

Universidade Estadual de Campinas
Instituto de Computação

Tarefa 4

Gradle, Javadoc e Tratamento de Erros

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis
PEDs: Athyrson Ribeiro, Matheus Lozano
PADs: João Pedro Frare



---

1. Descrição Geral

A planta industrial desenvolvida nas tarefas anteriores tornou-se um sistema mais completo, com múltiplas classes, estratégias de produção, interfaces, enumerações, cenários de operação, auditoria e gerenciamento de demandas.

Com o crescimento do projeto, novas necessidades surgem relacionadas à sua manutenção e utilização por diferentes desenvolvedores:

1. **Automatização do *build*:** o processo de compilação e execução não deve depender de comandos manuais que identifiquem individualmente os arquivos `.java`.
2. **Documentação do sistema:** as principais classes e operações devem possuir documentação padronizada, permitindo que outros desenvolvedores compreendam e reutilizem o código.
3. **Robustez:** operações inválidas e situações excepcionais não devem encerrar o programa de forma inesperada ou deixar o sistema em um estado inconsistente.

Nesta tarefa, o projeto deverá ser convertido para Gradle, documentado utilizando Javadoc e refatorado para tratar adequadamente erros e situações excepcionais.

A funcionalidade desenvolvida nas tarefas anteriores deve ser preservada. A Tarefa 4 não deve substituir as funcionalidades existentes, mas sim melhorar a forma como o projeto é construído, documentado e executado.

2. Objetivos

- Utilizar o **Gradle** para automatizar compilação, execução e outras tarefas do projeto;
- Compreender e aplicar uma estrutura de projeto compatível com o sistema de *build* adotado;
- Documentar classes, métodos e elementos públicos utilizando **Javadoc**;
- Compreender o uso de exceções para representar e tratar situações anormais;
- Diferenciar erros de entrada, erros de regra de negócio e falhas inesperadas;
- Implementar mecanismos de tratamento de erros sem comprometer o funcionamento normal da aplicação;
- Manter o código organizado, legível e coerente com a arquitetura desenvolvida nas tarefas anteriores.

3. Atividades

3.1. Conversão do Projeto para Gradle

O projeto desenvolvido nas tarefas anteriores deverá deixar de depender de comandos manuais como:

```
javac -d bin $(find src -name "*.java")
java -cp bin Main
```

e passará a utilizar o Gradle como ferramenta oficial de compilação e execução.

3.1.1. Estrutura do Projeto

O projeto deverá adotar uma estrutura compatível com um projeto Java utilizando Gradle. Como referência, espera-se uma organização semelhante a:

```
├── build.gradle
├── settings.gradle
├── gradlew
├── gradlew.bat
├── gradle/
│   └── wrapper/
├── README.md
├── justificativa.txt
└── src/
    └── main/
        └── java/
            ├── Main.java
            ├── GerenciadorProducao.java
            ├── Demanda.java
            ├── Maquina.java
            ├── Produto.java
            ├── EstrategiaProducao.java
            └── ...
```

A organização interna poderá ser modificada de acordo com a solução do grupo, desde que a estrutura seja coerente e o projeto possa ser construído e executado por meio do Gradle.

3.1.2. Configuração do Gradle

O projeto deverá possuir, no mínimo:

- arquivo `build.gradle`;
- arquivo `settings.gradle`;
- configuração do projeto como aplicação Java;
- configuração das dependências, caso existam;
- configuração necessária para compilação e execução do sistema.

O projeto deverá disponibilizar também o G**radle Wrapper**, permitindo que a aplicação seja construída sem exigir uma instalação específica do Gradle na máquina do avaliador.

3.1.3. Tarefas Esperadas

A configuração deverá permitir, no mínimo, a execução das seguintes operações:

- `./gradlew build`: para compilação e construção do projeto;
- `./gradlew run`: para executar a aplicação;
- `./gradlew javadoc`: para gerar a documentação Javadoc.

No Windows, deverão ser disponibilizadas as versões equivalentes utilizando `gradlew.bat`.
O projeto deverá ser capaz de ser construído a partir de um ambiente limpo, sem depender de arquivos compilados previamente ou de comandos manuais adicionais.

3.2. Documentação com Javadoc

Todo o código do projeto deverá ser documentado utilizando Javadoc. A documentação deverá permitir que outro desenvolvedor compreenda:

- a responsabilidade de cada classe;
- a finalidade de cada interface e enum;
- o comportamento dos principais métodos;
- os parâmetros recebidos;
- os valores retornados;
- as exceções que podem ser lançadas;
- as principais regras de negócio implementadas.

Os atributos privados não precisam necessariamente possuir Javadoc individual, mas seus significados deverão estar claros por meio da documentação da classe e dos métodos relacionados.

Os grupos deverão utilizar adequadamente, no mínimo, `@param`, `@return`, `@throws` e `@author`, podendo utilizar outras *tags* quando apropriado.

3.2.1. Geração da Documentação

Executar `./gradlew javadoc` deverá gerar a documentação Javadoc do projeto sem erros.

O código deverá permanecer documentado diretamente nas classes `.java`; não será suficiente produzir apenas um arquivo textual descrevendo o sistema.

3.3. Tratamento de Erros e Exceções

O sistema deverá ser refatorado para lidar adequadamente com situações de erro durante sua execução. O objetivo não é apenas evitar que o programa seja encerrado, mas identificar a situação problemática, informar o usuário e permitir que o sistema continue funcionando sempre que isso for possível.

O programa deverá tratar situações como, por exemplo:

- entrada inválida no menu;
- escolha de uma opção inexistente;
- tentativa de produzir uma demanda inexistente;
- tentativa de executar uma operação incompatível com o estado atual de uma demanda;
- falta de matéria-prima;
- orçamento insuficiente;

Entradas fornecidas pelo usuário deverão ser validadas. Os grupos deverão identificar outras situações de erro coerentes com as regras de negócio de sua implementação.

3.3.1. Exceções de Domínio

As situações relacionadas às regras da planta deverão ser representadas de forma clara por meio de exceções.

O grupo deverá criar **exceções próprias para erros relevantes do domínio**, em vez de representar todos os problemas apenas por mensagens impressas no terminal. Por exemplo, poderão ser criadas exceções como:

```
public class RecursoInsuficienteException extends RuntimeException {
    // ...
}
```

ou uma hierarquia de exceções de domínio:

```
PlantaException
├── RecursoInsuficienteException
├── DemandaInvalidaException
└── MaquinaIndisponivelException
```

Os nomes das exceções são livres, desde que sejam coerentes com o tema do projeto. Não é obrigatório utilizar exatamente a hierarquia acima.

3.3.2. Tratamento das Exceções

As exceções deverão ser tratadas no ponto adequado da aplicação. Por exemplo, se uma operação do sistema lançar uma exceção de negócio, a camada responsável pela interação com o usuário deverá:

1. capturar a exceção;
2. apresentar uma mensagem clara ao usuário;
3. preservar o estado consistente da aplicação;
4. permitir que o usuário escolha outra operação.

Exemplo:

```
========================================
ERRO
========================================
Não foi possível iniciar a produção:
matéria-prima insuficiente para fabricar
o lote solicitado.

Nenhuma alteração foi realizada no estoque.

Pressione ENTER para continuar...
```

Não será considerado tratamento adequado simplesmente fazer:

```
try {
    // ...
} catch (Exception e) {
}
```

ou capturar uma exceção sem informar o usuário ou corrigir adequadamente o fluxo da aplicação.

3.3.3. Consistência do Estado

Um erro durante uma operação não deverá deixar o sistema parcialmente modificado.

Por exemplo, se uma produção falhar por falta de recurso, não deverá ocorrer uma situação em que parte do estoque já tenha sido consumida, o orçamento já tenha sido alterado ou a demanda esteja marcada como concluída sem que a operação tenha realmente sido realizada.

A implementação deverá considerar a ordem das operações e garantir que situações de erro não produzam estados inconsistentes.

3.4. Integração com o Sistema Existente

Todas as funcionalidades desenvolvidas nas tarefas anteriores deverão continuar disponíveis após a migração para Gradle e a introdução do tratamento de erros.

O projeto deverá manter:

- gerenciamento das demandas;
- seleção de estratégia de produção;
- funcionamento das estratégias implementadas;
- consulta de matéria-prima;

- compra de matéria-prima;
- cenários de operação;
- demais regras de negócio implementadas pelo grupo.

A introdução de exceções não deverá substituir o comportamento normal do sistema.

Por exemplo, a ocorrência de uma exceção causada por uma tentativa inválida de produção não deve fazer com que o programa seja encerrado. Após o tratamento do erro, o usuário deverá poder continuar utilizando o sistema.

3.5. Menu e Interação com o Usuário

O menu desenvolvido na tarefa anterior deverá continuar funcionando após a migração para Gradle e deverá incorporar o tratamento adequado das entradas inválidas.

Não é necessário criar novas funcionalidades de negócio apenas para esta tarefa. Entretanto, o sistema deverá:

- informar claramente operações inválidas;
- evitar encerramentos inesperados;
- permitir a recuperação após erros;
- manter mensagens de erro compreensíveis para o usuário.

As mensagens deverão ser suficientemente descritivas para que o usuário compreenda o motivo pelo qual a operação não pôde ser realizada.

4. Avaliação

1. **Configuração e Utilização do Gradle (2,5 pontos)**
   - Estrutura correta de projeto Gradle;
   - Configuração adequada do `build.gradle`;
   - Configuração adequada do `settings.gradle`;
   - Funcionamento do *build*;
   - Presença e funcionamento do Gradle Wrapper;
   - Projeto construído sem necessidade de comandos manuais adicionais.
2. **Documentação com Javadoc (2,5 pontos)**
   - Classes, métodos, interfaces e enums devidamente documentados;
   - Utilização adequada de *tags*;
   - Documentação coerente com o comportamento real do sistema;
   - Geração correta da documentação pelo Gradle.
3. **Tratamento de Erros (4,0 pontos)**
   - Identificação adequada das situações de erro;
   - Criação de exceções de domínio quando necessário;
   - Tratamento apropriado das entradas inválidas;
   - Mensagens claras e úteis;
   - Continuidade da execução após erros recuperáveis;
   - Preservação da consistência do estado da planta;
   - Ausência de catch vazios ou tratamento genérico.
4. **Organização do Código (1,0 pontos)**
   - Código legível e coerente;
   - Nomes adequados;
   - Separação de responsabilidades;
   - Preservação das funcionalidades das tarefas anteriores;
   - Integração correta das novas funcionalidades com a arquitetura existente.
5. **Penalidade:** Não compilação = -5,0 pontos.

5. Entrega

1. **GitHub:** Release `tarefa4-RA1-RA2`;
2. **Google Forms:** Nomes, RAs e link da release.

O prazo de submissão é o dia **08/10/2026 às 23h59**. Entregas atrasadas serão penalizadas em **25 % da nota obtida**. Não serão aceitas entregas atrasadas após **15/10/2026**.

5.1. Compilação e Execução

```
./gradlew build
./gradlew run
./gradlew javadoc
```

6. Dicas de Boas Práticas

- **Gradle:** evite colocar lógica de negócio nos arquivos de configuração do *build*. O `build.gradle` deve ser responsável apenas pela configuração da construção do projeto.
- **Javadoc:** documente o comportamento e as regras importantes, não apenas o significado óbvio do nome do método.
- **Exceções específicas:** prefira exceções que expressem claramente o problema ocorrido.
- Validação: valide entradas e condições antes de executar operações que possam alterar o estado do sistema.
- **Recuperação:** sempre que possível, erros de interação devem retornar o usuário ao menu em vez de encerrar o programa.
- **Mensagens:** uma boa mensagem de erro deve explicar o que aconteceu e, quando possível, como o usuário pode corrigir o problema.
- **Não ocultar erros:** não utilize blocos `catch` vazios nem capture genericamente todas as exceções sem necessidade.
- **KISS e SRP:** não crie uma hierarquia excessivamente complexa de exceções. Utilize apenas as abstrações que realmente contribuam para a organização do sistema.
- **YAGNI:** Não implemente o que não é necessário.